

Performance study of a general purpose Bayesian Nonparametric Modeling software

Project report PCS951

Aurora Grefsrud

Last edited November 15, 2024

Introduction

Bayesian nonparametric statistics is an elegant mathematical formalism for getting an idea about the shape of the distribution of data scientists have gathered in an experiment. The theory for these kinds of calculations is quite simple. However, in most nontrivial cases it is not possible to calculate the Bayesian posterior distribution directly. We therefore have to resort to computational approximation algorithms such as Monte Carlo sampling. As we are sampling in extremely high dimensional space for all nontrivial cases, Monte Carlo sampling has to be guided by "importance sampling" algorithms (for instance a Markov Chain) to make the approximation converge in a reasonable amount of time. Our sampling method then becomes a full-blown simulation of one possible finite sampling scenario. To make sure that our results are not biased by the not-so-random-anymore-walk of the Markov Chain Monte Carlo simulations, we run many such simulations with different starting seeds. Running each Markov Chain Monte Carlo (MCMC) simulation requires one CPU core, so the more CPU cores we have, the faster we can finish our calculations. This makes MCMC simulations ideal for parallelization and HPC computing. This is what ATLAS spends the majority of its computational budget on, although they are not simulating Bayesian nonparametric posterior distributions, but instead they are simulating quantum mechanical processes and how they propagate through space and interact with the detector. A major criticism against using Bayesian inference in many physics experiments is that it is simply too computationally expensive.

In this project, the computational cost of a Bayesian nonparametric model simulation as a function of the number of input data points is studied. To do this efficiently, an analysis pipeline which spawns a series of multi-node tasks which will run for a variable amount of time, with different input arguments for each element, and records the results in a systematic way is set up. The computational setup and the benefits of parallelization is evaluated as part of the study. The software studied in this project is the beta-version of an R-package named Inferno ([link to GitHub](#)), which aims to simplify Bayesian Nonparametric population inference for scientists without the computational and statistical background to build their own MCMC simulations. It was developed by PierGianLuca Porta Mana at Western Norway University of Applied Sciences, and I have contributed to it as well, which is why I am especially interested in the performance of the software.

Probability density estimation

The most influential sources for this section are the textbooks "Probability Theory, The Logic of Science", written by E.T. Jaynes [1] and "Statistical Computing with R", written by Maria L. Rizzo [2].

In many scientific experiments, the underlying aim of the experiment is to find out something about the long term frequency density distribution of the measurable we are collecting data on. A frequency density distribution is an analytical function which represents the trend of how likely the measure is to take on certain values. In most of these studies, we suppose that the population the measurements are drawn from is "infinite". That is, we never run out of possible measurements to make. It is also supposed that each data point is randomly drawn from the density distribution, with a probability proportional to the height of the distribution at each possible measurement value. Examples could be repeated measurements of the gravitational constant, measuring the height of adults in Norway or measuring the number of days of a pregnancy. Some of these examples fit better than the others into the "infinite population" model, but they can all be analyzed with statistical methods based on those principles. The validity of the result of a statistical inference process will always be dependent on the discrepancy between model and reality. Typical statistical models for probability density distribution estimation are Gaussian distributions and Gamma distributions. These methods are so-called parametric distributions, because they are fully defined by a small set of parameters. Observational data can be used to choose the parameters so that the distribution fits the data. Bayesian statistics is a branch of statistics where we view the parameters of the model as uncertain variables, and the job of the statistician is to see how (and if) the parameters become more well-defined with the new data. This is opposed to frequentist statistics which view the parameters as completely well-defined, but unknown, and the observed data lets us become more certain about the interval within which the parameter value lies.

Bayesian Nonparametric Models

Another method for density estimation is nonparametric statistics [3]. A histogram is an example of a nonparametric density distribution model. In the case of this study, I use a more advanced method to produce a smooth multivariate probability density distribution estimate by using a model for the frequency distribution consisting of a large sum of parametric distributions, with the parametric family chosen according to the numerical type of the measurable in the data. The data frequency distribution is assumed to be infinite dimensional (in the case that it is a continuous variable), but the function representing it is defined on a finite-dimensional space, albeit very large. The software I am studying automatically adds the required complexity needed if the data becomes more complex by the increase of the number of variables. The goal of the inference process is to somehow estimate the probability density generating the data. There are several ways to do this, but the most common is to use Markov Chain Monte Carlo methods.

Markov Chain Monte Carlo

Inferno uses a Gibbs-sampling scheme, which is a kind of Markov Chain Monte Carlo. Simply explained, the Gibbs sampling works by randomly initializing the parameters of the model, generating a kind of test distribution, then evaluating the probability that the observed data came from the test distribution, and finally use this probability to generate new parameters from a prior distribution of

parameters. The sampler will converge towards the parameters that are more likely to generate the data. The important part is to make sure that the generated set of parameters are still considered "random" draws from their underlying distributions, even if they are dependent upon the previous draws. This makes this method quite different from typical deep learning optimization methods, which do not attempt to obstruct the path towards higher likelihood. The drawback of MCMC sampling as a way of maximizing likelihood of the data is that it takes a long time to compute. The positives are that it is much less prone to overfitting (where the density estimate basically becomes the observed density), and it also produces a measure of the uncertainty of the model, as each set of (good enough) parameter draws are saved and considered part of the model. Gibbs sampling loops over every data point once during each calculation of a set of parameters (a sample), so we expect the computational time to be a linear function of the number of data points. We do not however know the slope of the function, and this is crucial to estimate how long an analysis will take.

To compensate for the Markov Chain getting stuck in local minima (or probability islands as it is known in NPBI literature), several Markov Chains can be run at the same time using the software. The required number of samples are then spread over the chosen number of chains, and each chain calculates until some convergence criteria is met. This should in theory increase the reliability of the result. Each such process usually requires a CPU to run (if the CPU has multithreading enabled, then each thread can support one chain), which is a limiting factor for most normal computers. However, it makes the software perfect for distributed computing. The behavior of these chains is a crucial part of this study.

Experiment

The project consists of a quantitative analysis of the performance of the software, as well as a qualitative analysis of how well the different distributed computing software I used aided me in the study. The software performance study is quite straightforward:

1. Generate data for the software to process in an analysis
2. Make a grid of number of data points to include in the analysis
3. Run the software for each grid point
4. Record information such as computational time for each point
5. Gather the data from the experiment
6. Analyze the data to evaluate the performance of the software

Data

The data for this project is samples of the variables x_1, x_2 generated from a multivariate toy distribution with distributed parameters r_1 and a_1 . The details of the distribution are not relevant for the project, but the scientific aim of the analysis is to see to what degree the software can reproduce the long run frequency distribution of the generating process. For this project we only care about the performance of the software with respect to computational parameters. To train the model, 5000 samples are generated from the distribution. These are presented in Figure 1

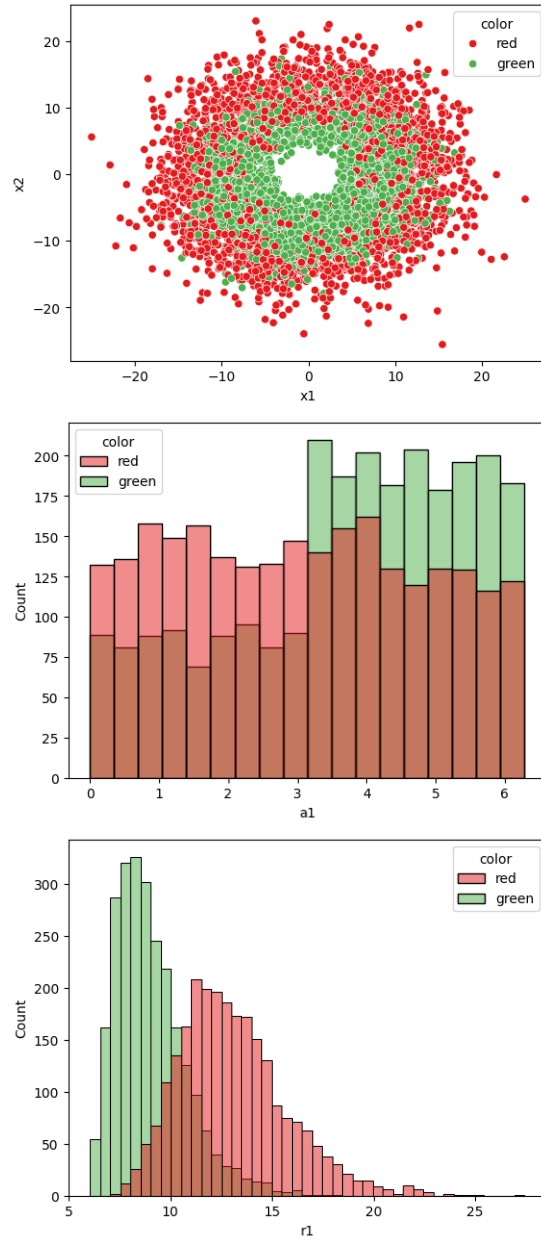


Figure 1: The top figure shows the training data as a scatter plot. The middle figure shows the distribution in the parameter a_1 . The bottom plot shows the distribution in the parameter r_1

NPBI analysis

The software is written in R, and so I have written a simple R script with input arguments which will run my analysis and some tests. This file is named “inferno_analysis.R”. It takes 2 positional, mandatory input argument and up to four optional arguments. See the entire file in Appendix.

The positional arguments are the path to the data files, while the optional arguments are the number of training data points which we want to investigate, as well as some options for the MCMC simulations and parallelization.

The analysis produces a folder with the output of the MCMC simulation, some logs with information on the computational time and other MCMC statistics, as well as an hdf5 file with the output of the probability calculations of my test set. The script can be run from the terminal using the command “Rscript”.

```
Rscript inferno_analysis.R -N 128 -c 20 -p 20 -s 1200 train.csv test.csv
```

A python script parses the log files to gather the relevant statistics from the different Markov Chains in each run. For this study, the focus is on the computational time and the number of MCMC iterations.

User experience analysis

The qualitative analysis of how well the different software used for the grid analysis performed are compared to a baseline with no such tools. Without a job scheduler, I would have to run each grid point manually, possibly making sure that the ssh connection did not disconnect during the run time of the program. I would have to check in periodically to see if the program finished running. This is how I have run such tasks in the past, while getting acquainted with the software. Another big problem I have faced is that of not having full control of my software environment. The code is written in R, and installing R locally with a non-privileged user is not ideal. I have wasted many days trying to set up software environments that let me compile and install the necessary software, and in general this is very frustrating. Even with CVMFS this can quickly become a hassle.

Computational setup

Cluster

The experiment will be run on the HVL ATLAS cluster. The computing system consists of three workstations, atlas[1,2,3]. The operating system is Linux CentOS 7.

- atlas1 Dell order 28 GB RAM 24x Intel(R) Xeon(R) 64 bit CPU @ 2.4 GHz (2x6 cores) 8x2 TB local disks hosts the home directory for all users
- atlas2 64 GB RAM 40x Intel(R) Xeon(R) 64 bit CPU @ 3.0 GHz (2x10 cores) 4x3 TB local disks 1 GPU: GeForce RTX 2080 Ti
- atlas3 128 GB RAM 48x Intel(R) Xeon(R) 64 bit CPU @ 3.0 GHz (2x12 core) 5x4 TB local disks 1 GPU: NVIDIA RTX A4500

To keep the impact of this study minimal for other users, only the atlas2 node is used.

Containerization

Because we don't install a lot of software on the cluster, the most convenient way to have a safe environment for our project is to use containerized images. I use Apptainer [4] with a Docker image called "r-base" and install the relevant R-packages. I have created a definition file which lets me easily build the image, and I have added the line of code which executes the R-script with input arguments from the `apptainer run`-command. To install the custom R package, a link to the locally downloaded github repo which contains the software library must be added, as a user within Apptainer does not automatically access all the files of a user outside it. This is part of the reason why Apptainer is seen as a very safe containerization software.

To build the container:

```
apptainer build inferno\_env.sif apptainer.def
```

To install the software inside the container:

```
Rscript -e "install.packages(pkgs='/disk/atlas2/users/agrefsru/inferno\_renegade',  
  repos=NULL)"
```

To run the container:

```
apptainer run --bind /disk/atlas2/users/agrefsru/inferno_renegade inferno\_env.sif  
  5 1 1 100 data/train.csv data/test.csv
```

Job scheduler

A TORQUE [5] batch system has been set up to share CPU resources among the local computers of the HVL ATLAS cluster. TORQUE is managed by Adaptive Computing, Inc [AdaptiveComputingInc]. Instructions on how to use TORQUE can be found in Adaptive Computing, Inc TORQUE Administrator guide. When jobs are submitted, the scheduler decides to send them either to atlas1 / atlas2 / atlas3, depending of the available resources of each machine at that time. The usage of the cluster is monitored with a Ganglia-web interface, which allows the administrator of the cluster to detect problems before they become unmanageable. It is also a convenient way to check if resources are free. The queuing system is quite simple because the users are all known and can communicate easily. There is a "short queue" with a maximum wall time of 72 hours per job, and an "unlimited" queue with no time out. When issuing the `qsub` command, the system returns a `JOB_ID`, for instance 142.atlas2. When monitoring/managing jobs, the useful part of the `JOB_ID` is the number in front, for instance `qdel 142` would kill the job. Once the job is finished (failed or successful), an output log file and an error log file is created in the directory from where the job was submitted. Their default name should be, in the example case here, `my_script.sh.o.142` and `my_script.sh.e.142`, respectively.

Experimental results

The total computational time as function of number of data points is shown to be a linear function. The flexibility of the setup and the ease of repeating the experiment with just a few changes allowed

me to trial a few different settings of the software, and add some more complex data. The results are presented in Figure 2. The results show that the computational time is approximately linear as a function of number of data points. Doubling the number of continuous features (input variables) to the data set increased the computational time by a factor of 3. Using all the data points to test for convergence, as opposed to the default of the number of input features plus two, increased the computational time by a factor of 8. However, increasing the number of test points also made all the test statistics much smoother. It is probably a good idea to increase this number to decrease stochastic erratic behavior of the algorithm.

Another interesting result obtained is the minimum, maximum and mean number of Monte Carlo iterations and computational time required for one of the Markov Chains to reach completion. Figure 3 shows the spread in these variables. There is a huge discrepancy between the chain requiring the most and the least amount of time. This provides a way that distributed computing using job schedulers can improve the computational calculation time if several analyses are run at the same time. If the job scheduler can free up the nodes as they complete and start a new job, the entire process would be accelerated. This would be a topic for further study.

Evaluation of computational setup and discussions

Using Torque greatly enhances the user experience of doing a grid-job style analysis. First, the fact that jobs can be started and then the user can log off and do something else without having to worry about keeping the ssh connection open, makes everything much more efficient. Secondly, it was easy to rerun jobs if something was wrong in one of the scripts, or some enhancements were made to the analysis. Being able to delete the array of jobs if something is wrong is also good, and Torque makes this much easier. The varying length of MCMC chain computational times makes this kind of analysis especially well suited to flexible parallelization schemes.

Torque's job description system is flexible, but it is not ideal. Documentation is quite bad, and it is hard to combine the job description comments with normal bash scripting (at least the barrier to entry seems high). Other job schedulers like HTCondor might be even more flexible and easy to use. Unfortunately, in this project there was no time to try different job schedulers, so this is something I must explore further in the future.

The part of my project that really should be improved is better handling of semi-constant parameters such as the location of the R-script metadata file, the folder names, etc. The system I use now lets me update these in the R-script, but ideally it would be nice to add them as arguments. However, this requires rebuilding the Apptainer container, which takes approximately 20 minutes because a lot of R-modules are installed. This issue could be mitigated by rebuilding the container from the already made container, but I had no time to learn how to do that.

In conclusion, the project enabled me to perform an interesting performance study of the software I am using. By employing the strategies learned in the PCS951 course, I was able to improve my workflow greatly. The setup also allowed the computation of the analysis to go much faster, and use computational resources more efficiently.

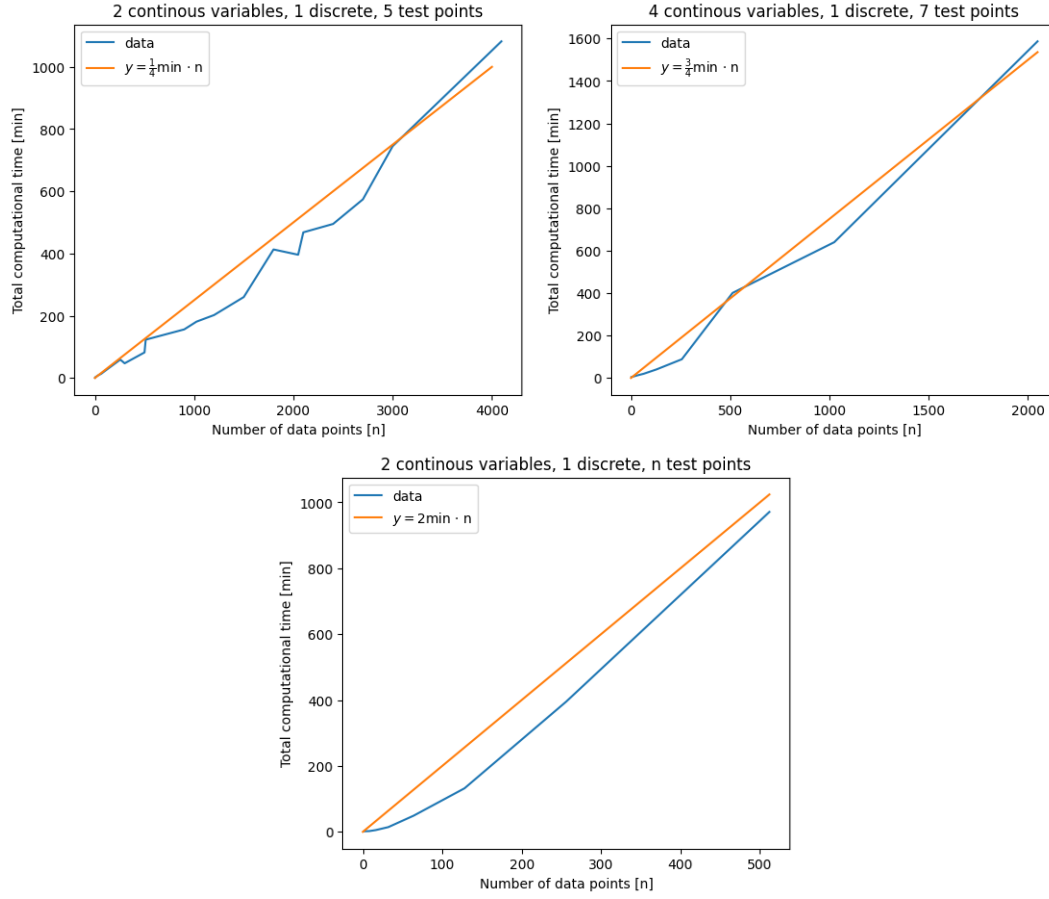


Figure 2: The two plots on the top shows the computational time in minutes as a function of the number of data points in the training data (blue line), with the left plot being the results for a data set with 2 continuous variables, and the right plot the same data set with 4 continuous variables. The bottom plot shows the results when all the data points are used for evaluating the convergence of the model. All plots also include a linear function to indicate the steepness of the slope.

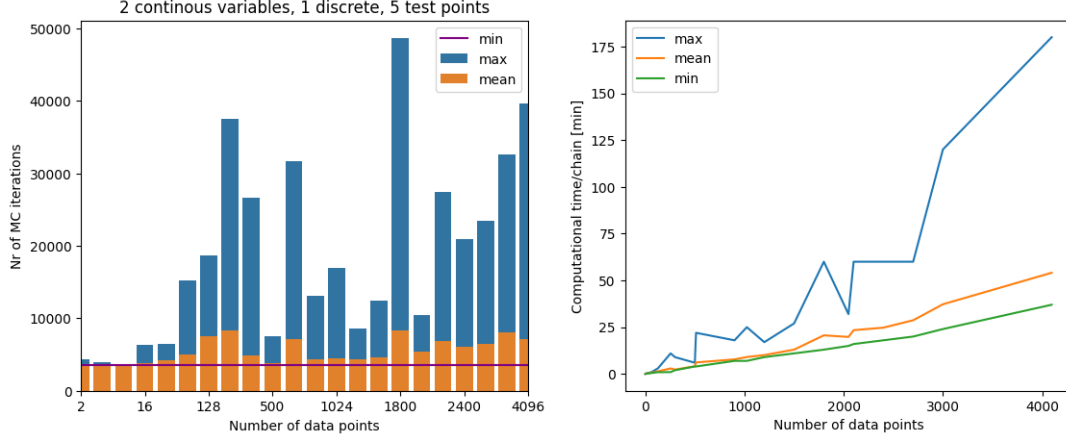


Figure 3: On the left: Histogram showing the number of Monte Carlo iterations before convergence as a function of data points. The blue bars represent the maximum nr of iterations for each point, the orange the mean, and the purple line shows the minimum nr of iterations as required by the software settings. On the right: The computational time per chain as a function of number of datapoints. The blue line shows the maximum amount of time of the set of chains, the green line shows the minimum amount of time and the orange line shows the mean.

References

- [1] E. T. Jaynes. *Probability theory: The logic of science*. Cambridge: Cambridge University Press, 2003.
- [2] M.L. Rizzo. *Statistical Computing with R, Second Edition*. Chapman & Hall/CRC The R Series. CRC Press, 2019. ISBN: 9780429527760. URL: <https://books.google.no/books?id=kSeNDwAAQBAJ>.
- [3] P. Müller and R. Mitra. “Nonparametric Inference - Why and How”. In: *Bayesian Anal.* (2013). DOI: 10.1214/13-BA811. URL: <https://doi.org/10.1214/13-BA811>.
- [4] Apptainer project and Sylabs Inc. <https://apptainer.org/docs/user/main/index.html>.
- [5] Inc. Adaptive Computing. <https://adaptivecomputing.com/>.

Appendix

inferno_analysis.R

```
library("inferno")
library('rhdf5')
library('optparse')

#####
# SETUP
#####

# MAX_N <- 4000 # maximum number of training points

# We want to sample the same N-1 datapoints for each increase in N so we set a seed
#sset.seed(1)
#datapoint_list <- sample(1:MAX_N, size = MAX_N, replace = FALSE)

# This script can be used with an argument specifying the number of data points to
#   train the model oncores
# It should be used with the path to the training data and the test data
# Read and parse arguments

parser <- OptionParser(usage="%prog [options] trainfile testfile")

parser <- add_option(parser, c("-N", "--trainpoints"),
  type = "integer", default = 5,
  help = "Number of data points to train on [default %default]")

parser <- add_option(parser, c("-c", "--chains"),
  type = "integer", default = 1,
  help = "Number of MCMC chains [default %default]")

parser <- add_option(parser, c("-p", "--cores"),
  type = "integer", default = 1,
  help = "Number of cores to use for parallel processing [default %default]")

parser <- add_option(parser, c("-s", "--samples"),
  type = "integer", default = 120,
  help = "Number of MCMC samples to generate [default %default]")

args <- parse_args(parser, positional_arguments = 2)
opt <- args$options

# Read number of datapoints
ntrain <- opt$trainpoints
nsamples <- opt$samples
nchains <- opt$chains
ncores <- opt$cores
#metadata <- opt$metadata
```

```

metadata <- "metadata.csv"

# Read files
traindatafile <- args$args[1]
testdatafile <- args$args[2]

train_df <- read.csv(traindatafile)
# Data is already shuffled, no need to reshuffle
traindata <- train_df[1:ntrain, ]
test_df <- read.csv(testdatafile)

# Create folder
inferno_dir <- paste0("analysis/", sub('\.csv$', '', basename(traindatafile)),
                      "-nsamples_", nsamples, "_ndata_", ntrain)
if(!dir.exists(inferno_dir)) {
  cat(paste0("Creating dir ", inferno_dir, '\n'))
  dir.create(inferno_dir, recursive=TRUE)
}

#####
# TRAIN MODEL
#####

# Start inference
learnt <- learn(
  data = traindata,
  metadata = metadata,
  outputdir = inferno_dir,
  nsamples = nsamples,
  nchains = nchains,
  parallel = ncores,
  appendtimestamp = FALSE,
  appendinfo = FALSE
)

#####
# RUN TEST
#####

# Classes
labels <- cbind(color = c("red", "green"))
nlabels <- length(labels)
xvalues <- test_df[c("x1", "x2")]
ntest <- dim(xvalues)[1]
if (nsamples > 100) {
  nsamples_save <- 100
} else {nsamples_save <- nsamples}

Pr_output <- Pr(Y = labels,
                X = xvalues,

```

```

        learnt = inferno_dir,
        nsamples = nsamples_save,
        parallel = FALSE,
        silent = FALSE)

condfreqs <- Pr_output$samples
quantiles <- Pr_output$quantiles
# Reshape the array from (nlabels, ntest, nsamples) to (nlabels, nsamples, ntest)
condfreqs <- aperm(condfreqs, c(1, 3, 2))
# Reshape the array from (nlabels, ntest, quantiles) to (nlabels, quantiles, ntest)
quantiles <- aperm(quantiles, c(1, 3, 2))

#####
# SAVE RESULTS
#####
# Create hdf5 file in outputdir
h5file <- file.path(inferno_dir, paste0('test.h5'))
# Overwrite if it already exists
if (!file.exists(h5file)) {
  h5createFile(h5file)
} else {
  file.remove((h5file))
  h5createFile(h5file)
}

# When these files are read in a C-program, the dimensions will be inverted
h5createDataset(h5file, 'probabilities', dims = c(nlabels, ntest))
h5createDataset(h5file, 'quantiles', dims = c(nlabels, 4, ntest))
h5createDataset(h5file, 'samples', dims = c(nlabels, nsamples_save, ntest))
h5createDataset(h5file, 'data', dims = c(nlabels, ntest))
h5createDataset(h5file, 'truth', dims = c(ntest))

# Write to file
cat('Writing to file \n')
h5write(Pr_output$values, file = h5file, name = 'probabilities',
        index = list(NULL, NULL))
h5write(quantiles, file = h5file, name = 'quantiles',
        index = list(NULL, NULL, NULL))
h5write(condfreqs, file = h5file, name = 'samples',
        index = list(NULL, NULL, NULL))
h5write(t(test_df[c("x1", "x2")]),
        file = h5file, name = 'data', index = list(NULL, NULL))
h5write(t(test_df["color"]), file = h5file, name = 'truth',
        index = list(NULL))

#For testing, let's plot the first test point to see that we can reproduce it in python

pdf(paste0(inferno_dir, "/test_plot.pdf"))
plot(sapply(Pr_output, '[', 1))

```

apptainer.def

```
BootStrap: docker
From: r-base

# Image for running the tests in inferno

%environment
    export R_LIBS_USER=./R-packages

%post
    # Install dependencies
    apt-get update -y
    apt-get install curl -y
    Rscript -e "install.packages('iterators')"
    Rscript -e "install.packages('numDeriv')"
    Rscript -e "install.packages('pracma')"
    Rscript -e "install.packages('coda')"
    Rscript -e "install.packages('igraph')"
    Rscript -e "install.packages('rngtools')"
    Rscript -e "install.packages('foreach')"
    Rscript -e "install.packages('nimble')"
    Rscript -e "install.packages('khroma')"
    Rscript -e "install.packages('extraDistr')"
    Rscript -e "install.packages('doRNG')"
    Rscript -e "install.packages('doParallel')"

%runscript
    #!/bin/bash
    # Input arguments must be in the order
    # N, ncores, nchains, nsamples, trainfile, testfile
    Rscript inferno_analysis.R -N $1 -c $2 -p $3 -s $4 $5 $6
```

my_job.sh

```
## Choose the node to run on
#PBS -l nodes=atlas2.driftslab.hib.no-0:ppn=20
## Name the analysis
#PBS -N InfernoJob
## Choose queue
#PBS -q unlimited
## Array of jobs
#PBS -t 1-11
## Concat output files
#PBS -j oe

nlist=(2 4 8 16 32 64 128 256 512 1024 2048)
cd /disk/atlas2/users/agrefsru/inferno_analysis
apptainer run --bind /disk/atlas2/users/agrefsru/inferno_renegade inferno_env.sif \\\
```

```
${nlist[${PBS_ARRAYID}-1]} 20 20 1200 data/train_n_5000kr7_kg3_s1_a1_highlow.csv \\  
data/test_n_800kr7_kg3_s1_a1_highlow.csv
```
